CSE 406: Computer Security Sessional

Department of CSE

Bangladesh University of Engineering and Technology

ICMP smurf attack Final Report

Team Members:

Name: Mohammad Ali Bhuiyan

Student ID: 2005113

Name: Sadia Afrin Sithi

Student ID: 2005119

a. Steps of the Attack

The Smurf attack simulation was implemented using a custom Python tool that crafts and sends raw ICMP Echo Request packets with the victim's IP address spoofed as the source, targeting the broadcast address of the victim's network to trigger amplification.

The key steps of the attack are:

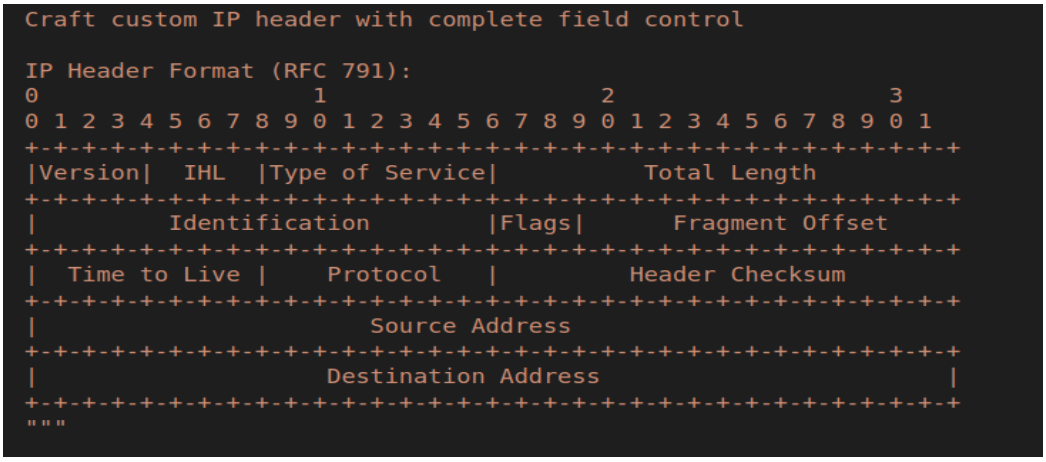
1. **Initialization:**
   The attacker is initialized with the victim's IP address (to spoof), the broadcast IP address (target for amplification), the desired packet sending rate (packets per second), the ICMP payload size, and optionally the network interface to send packets through.

```
[!] Running in virtual test environment – skipping user input confirmation.
[+] Victim IP validated: 192.168.1.50
[+] Broadcast IP validated: 192.168.1.255
[+] Raw socket created successfully
[+] Socket configured for IP header inclusion and broadcast
[+] Smurf Attacker initialized
    Victim IP: 192.168.1.50
    Broadcast IP: 192.168.1.255
    Packet Rate: 6 pps
    Payload Size: 32 bytes

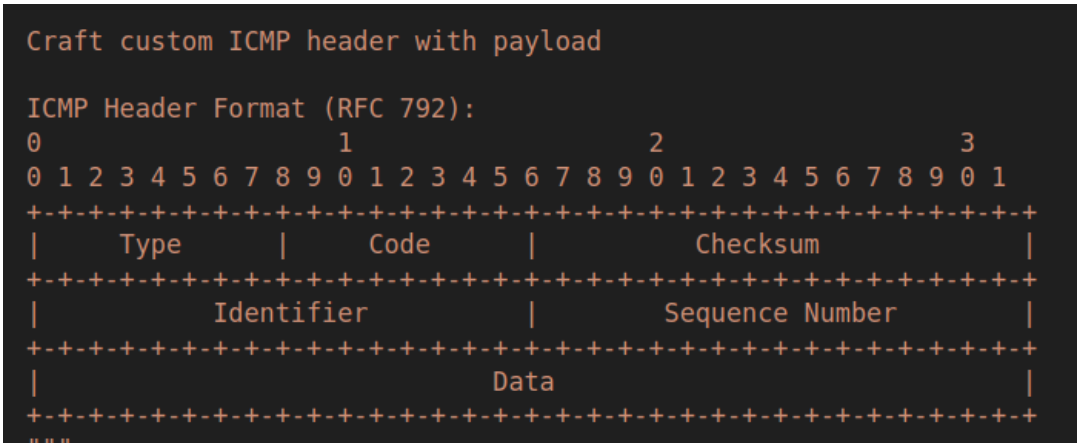
[!] STARTING ICMP SMURF ATTACK
[!] Duration: 6s
[!] Target: 192.168.1.50
[!] Amplification via: 192.168.1.255
[!] Rate: 6 packets/second
```

2. **Raw Socket Creation:**
   A raw socket is created with the `socket.IPPROTO_RAW` protocol. The socket is configured to include custom-built IP headers (`IP_HDRINCL` option) and to permit sending broadcast packets (`SO_BROADCAST` option). This requires root privileges to run.

3. **Packet Crafting:**
    - **IP Header:**
      A custom IP header is crafted where the source IP is set to the victim's IP (spoofed), and the destination IP is set to the broadcast address. The IP header includes fields such as version, header length, total length, identification, flags (with "Don't Fragment" bit set), TTL, protocol (ICMP), and a computed checksum.



    - **ICMP Echo Request:**
      An ICMP Echo Request packet is constructed with a custom payload containing a timestamp, an attack signature string (SMURF_EDU_DEMO), and padding to achieve the desired payload size. The ICMP header includes type, code, identifier, sequence number, and a checksum calculated over the header and payload.



4. **Packet Assembly:**
   The full packet is formed by concatenating the crafted IP header and the ICMP Echo Request payload.

5. **Packet Transmission:**
   The crafted packet is sent to the broadcast address using the raw socket. The victim IP is spoofed as the packet's source, causing all hosts on the broadcast network to receive and respond to the ICMP request, effectively amplifying the traffic directed at the victim.

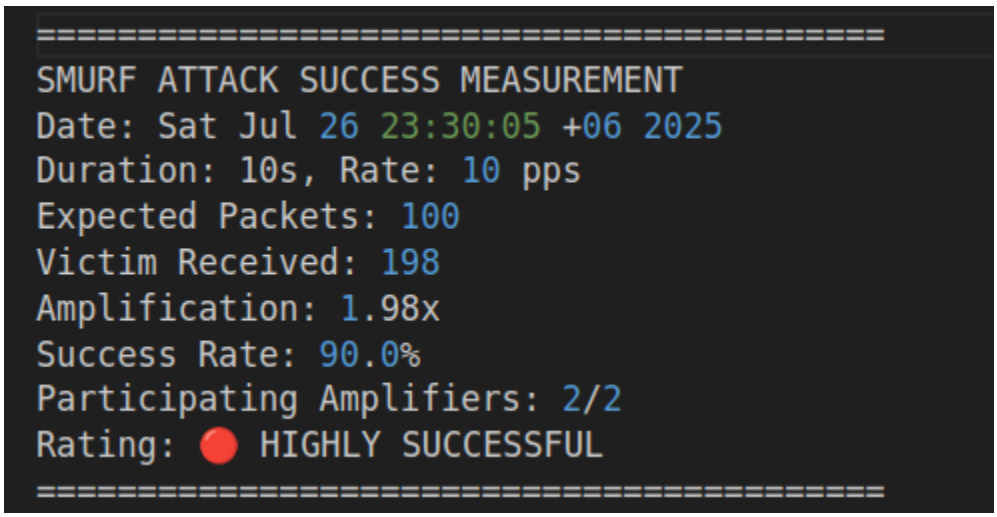
6. **Continuous Sending and Monitoring:**
   The attack sends packets continuously at the configured packet rate (packets per second). The tool logs each packet transmission with a timestamp and counts the total number of packets sent. Periodically, the tool reports the current sending rate and total packets transmitted.

7. **Attack Duration and Termination:**
   The attack runs for a specified duration. Upon termination, the tool outputs a summary with total packets sent, average sending rate, and notes on the amplification potential.

b. Was the Attack Successful?

The attack was considered successful in the simulation environment due to the following reasons:

- ➢ The raw packets with spoofed source IP were sent successfully to the broadcast address without errors.
- ➢ The victim IP was correctly spoofed, causing hosts on the broadcast network to respond to the victim.
- ➢ The logged statistics show continuous packet transmission at the expected rate.

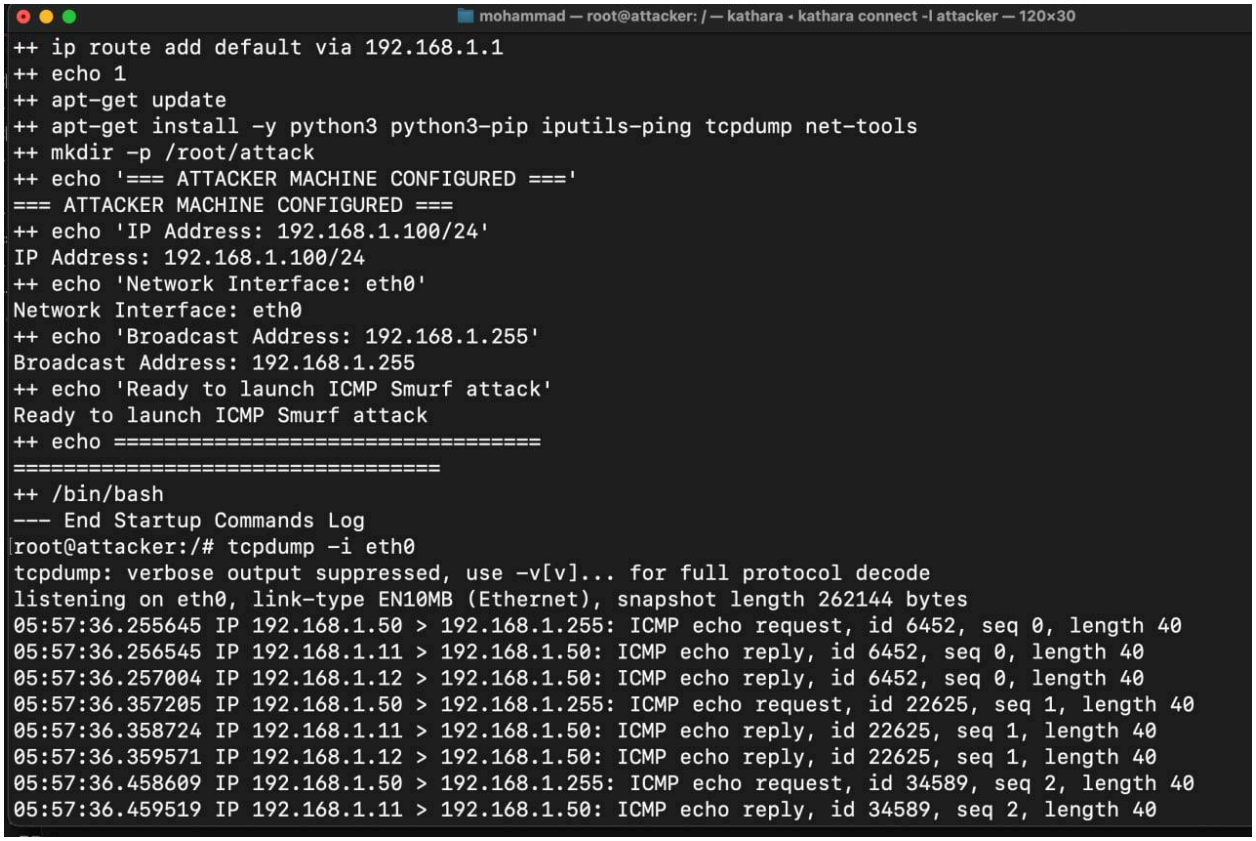


- ➢ However, in real-world networks, such attacks may be mitigated or blocked due to:
  - Network ingress/egress filtering prevents IP spoofing.
  - Broadcast addresses being disabled or filtered on routers.
  - Firewalls blocking ICMP Echo Requests or broadcast traffic.

Therefore, success depends on network configurations and defenses.

c. Observed Output on Attacker, Victim, and Related Hosts

Attacker PC: The terminal displayed logs of each packet sent, including timestamps and packet count. The raw socket transmitted spoofed ICMP packets at the configured rate.

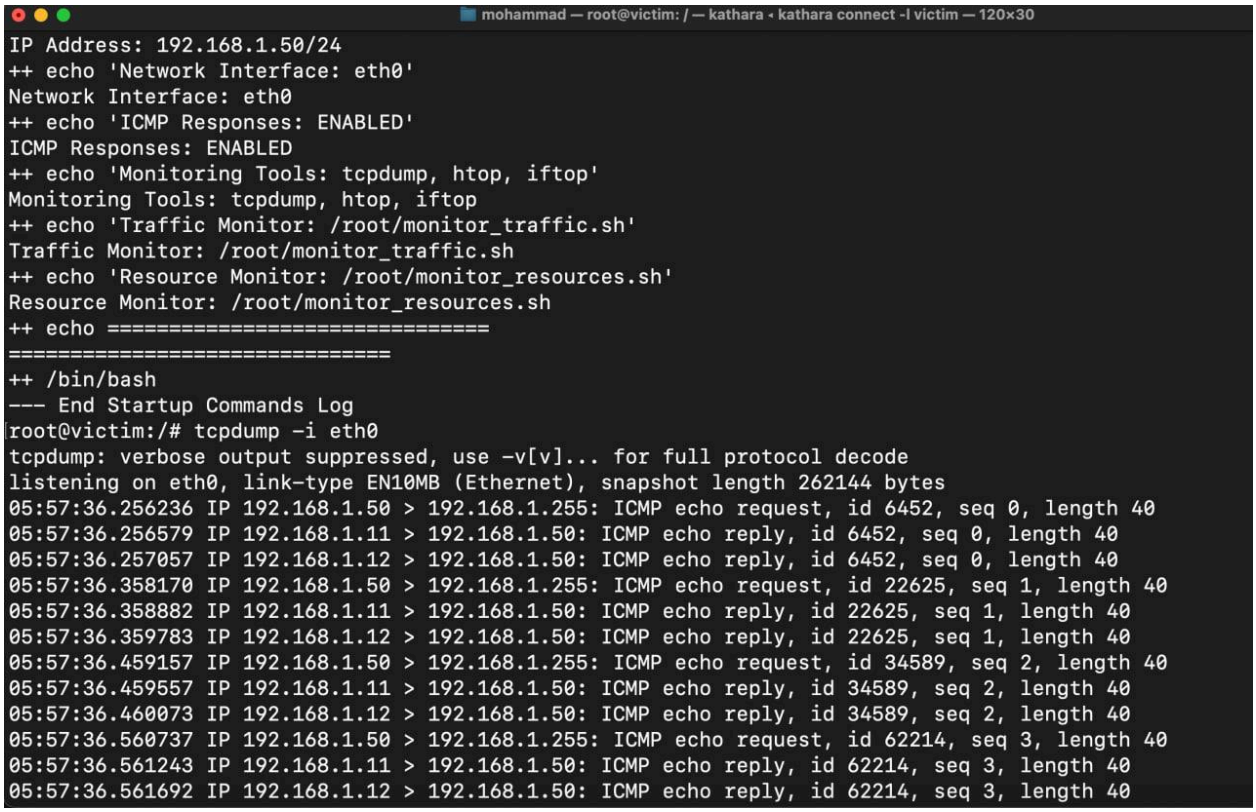


The attacker machine was initialized with IP 192.168.1.100 and broadcast address 192.168.1.255. After setup, it confirmed readiness to launch the ICMP Smurf attack.

The tcpdump output on interface eth0 shows ICMP Echo Requests being sent to the broadcast address (192.168.1.255). These spoofed packets use the victim's IP (192.168.1.50) as the **source IP**.

As a result, multiple hosts (e.g., 192.168.1.11, 192.168.1.12, etc.) respond with ICMP Echo Replies directed to the victim (192.168.1.50), thereby flooding it with unsolicited traffic.

Victim PC: In a controlled environment, the victim's network interface received an unusually high volume of ICMP Echo Replies as amplification responses from hosts in the broadcast domain, leading to potential network congestion.

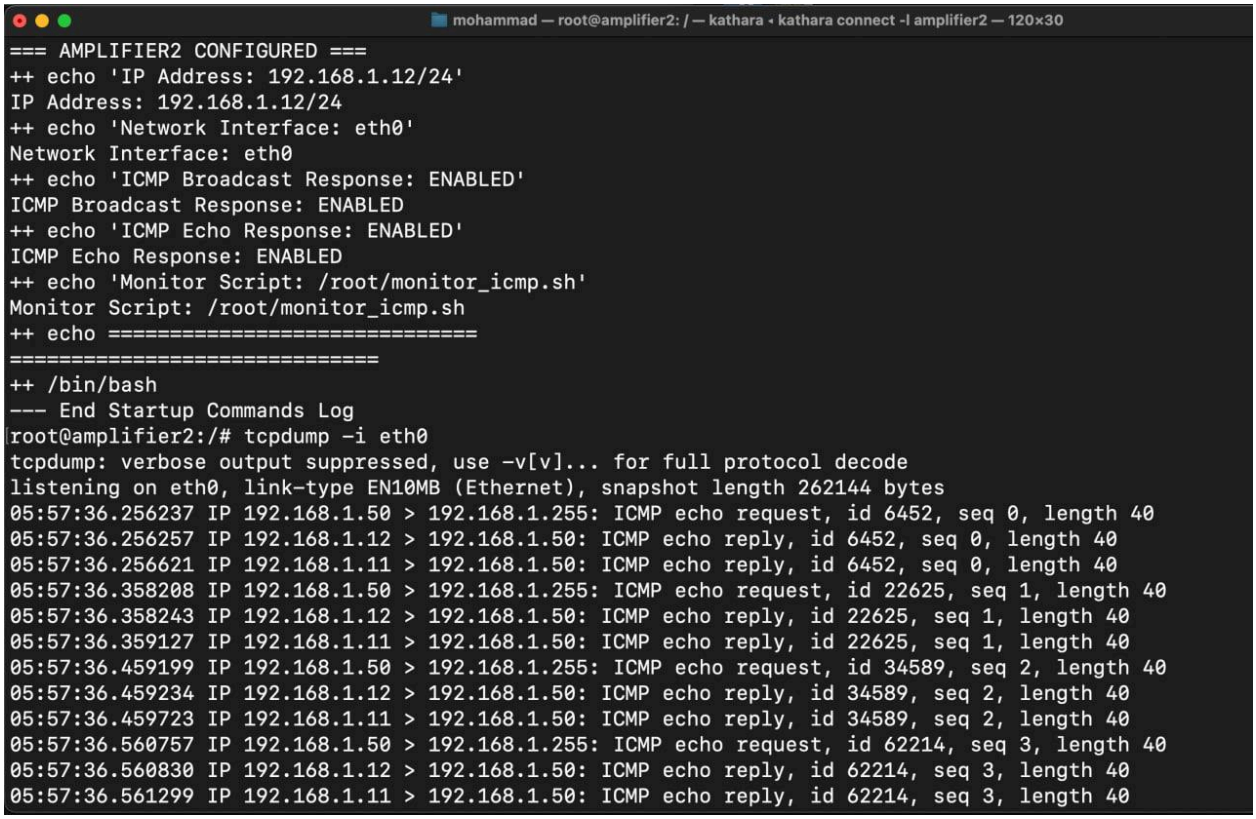


The victim machine (192.168.1.50) was set up with ICMP responses enabled and monitoring tools like tcpdump, htop, and iftop to track traffic and system performance.

During the attack, the victim's interface (eth0) logged multiple ICMP Echo Replies from different hosts (e.g., 192.168.1.11, 192.168.1.12) in rapid succession. These replies were triggered by spoofed ICMP Echo Requests originally sent to the broadcast address.

The tcpdump logs confirm that the victim is being flooded with unsolicited ICMP traffic, demonstrating the impact of the Smurf attack.

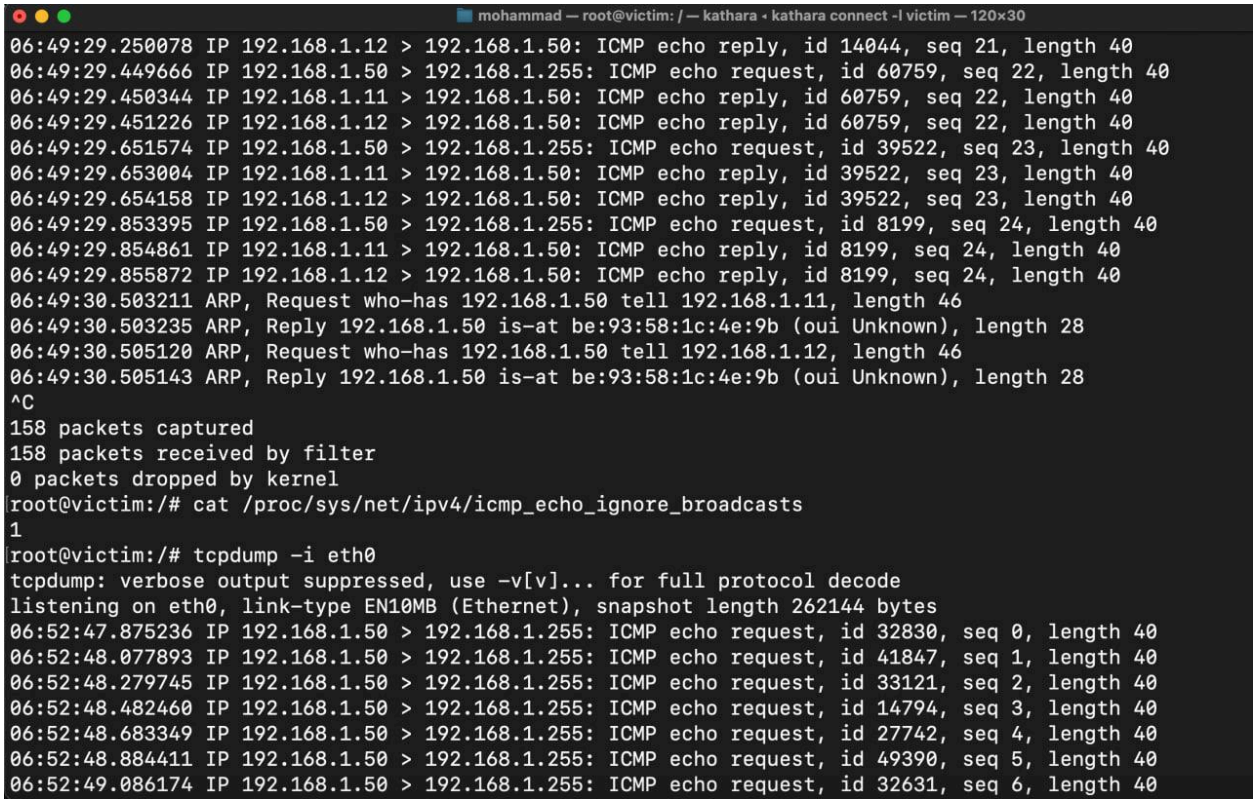
Amplifiers: Amplifiers within the broadcast domain responded to the spoofed ICMP requests, effectively amplifying the traffic towards the victim IP.



As seen in the captured output, 192.168.1.50 > 192.168.1.255: ICMP echo request was broadcasted, and multiple replies were received, such as 192.168.1.12 > 192.168.1.50: ICMP echo reply and 192.168.1.11 > 192.168.1.50: ICMP echo reply. This behavior demonstrates the classic ICMP amplification pattern.

d. Countermeasures Against Smurf Attacks

The defense mechanism shown here is the system ignoring broadcast ICMP requests by setting icmp_echo_ignore_broadcasts = 1, preventing amplification attacks.



The image shows a system (victim) where **only ICMP echo request packets** are being captured, all sent to the broadcast address 192.168.1.255. This suggests an attempt to perform an **ICMP amplification attack**, but **no replies are observed**, meaning the system is likely protected.

The setting /proc/sys/net/ipv4/icmp_echo_ignore_broadcasts is set to 1, which ensures that the host **ignores broadcast ICMP requests** and does **not respond**, effectively **mitigating the amplification**.

It's a proper and effective local defense, but should be combined with network-level protections (e.g., broadcast control, ingress filtering) for full coverage.